

AnimXML_Documentation

- [Change History](#)
- [1 Introduction](#)
- [2 Main Tag Structure](#)
- [3 Tag Breakdown](#)
 - [3.1 Component Tag](#)
 - [3.1.1 <assetreference>](#)
 - [3.1.2 <logic>](#)
 - [3.1.3 <attach>](#)
 - [3.1.4 <submodelref>](#)
 - [3.2 Anim Tag](#)
 - [3.3 Attachment and Submodel Tags](#)
- [4 Animation Types](#)

Document Version: 1.2

Change History

Version	Changes
1.1	Added popcornfxBloodFootprintName as an optional property when specifying popcorn effects. Added AssetPack and DLCTheme as valid logic types.
1.2	Added padVisualBoundsMinXYZ and padVisualBoundsMaxXYZ as attributes for Anim nodes.

1 Introduction

Anim XML files are all *.xml files located in the game/art directory. These define the relationship between animations and models. This also includes logic to determine when specific models or different animations should become enabled based on many unit or game state attributes.

Anim XML files are interpreted in the following way.

- <anim> tags define what animations are available. These usually pair against specific names to allow generalization. For example, "RangedAttack" for an animation that would be used for attacking from range.
- The <anim> tags provide information about the animation to be played and designate a model component for the animation to be played on.

- The models referenced by the animation should be defined first in the files; however, when necessary, they can also be referenced in supplementary Anim XML files.
- Models can have logic that controls the composition based on factors such as age or random variations.

The Sim Database is mentioned several times during this document. For modders, you may not need to use the Sim Database-specific Anim XML features. However, if you are generating custom Anim XML files that are directly referenced by the game, you may wish to make use of those tags. The Sim Database is the version of all data allowed to be known by the game simulation. This data is used to determine collisions, durations of attacks, which effects require spawning, and other sync-sensitive events for replays. The Sim Database is covered as a separate document.

2 Main Tag Structure

All Anim XML files are only considered valid if wrapped with **<animfile>** tags.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <animfile validation="strict">
3   <!-- model components -->
4   <!-- anims -->
5 </animfile>
```

3 Tag Breakdown

The following collection of tags is valid to include in the top-level wrapped with **<animfile>** around them. Each has a separate header following this table to further define relevant properties and proper structure.

Tag	Description
component	Defines a visual. This is usually a model, but can include particle effects and other types of visuals. Components can be named so anim nodes can reference them.
anim	Defines what animation is to be played, what visual to use, and any other attributes relevant to providing the visual state during the animation.
attachment	Attachments are additional models that can be combined into another component or anim. For example, combining cloth and vfx onto a ship in

	situations that can't be done through a composite model.
submodel	Submodels provide a way to define a model once and then reuse the same definition multiple times through variation logic.

3.1 Component Tag

The following table represents valid tags nested inside a component tag. For lookup, the name of component tags is the text that starts at the beginning of the tag.

```
1 <component>COMPONENTNAMEHERE
2   <!-- other tags here as defined below -->
3 </component>
```

Tag	Description
assetreference	Asset references can point to models, VFX, or any other valid asset file. A separate section below describes the valid tags you can include inside an assetreference tag.
logic	Controls variable state of the visual depending on factors like the current age or culture of the player.
attach	Attach references to some other named attachment in the XML file. Attachments are typically additional parts included on top of a main visual.
submodelref	References to a named submodel that has been defined in the XML file. This can be used to reuse the same model or split logic into more straightforward to understand parts.
nullvisual	Intentional flag to indicate there is supposed to be nothing shown in this situation. This does nothing when loaded by the model system, but is important for tagging intended control flow for the Sim Database. This type tag should be used when the other logic branches are visuals that are NOT attachments . It is a helper tag for the Sim Database.

nullattach	Intentional flag to indicate there is supposed to be nothing shown in this situation. This does nothing when loaded by the model system, but is important for tagging the intended control flow for the Sim Database. This type of tag should be used when the other logic branches are attachments . It is a helper tag for the Sim Database.
------------	---

3.1.1 <assetreference>

Asset references have a defined type and will change behavior based on the defined type. An example is shown below of the type attribute.

Attribute	Description
type	Type is a required attribute! This is required to identify the type of asset. Anything without a type specified will be ignored. For example, seen below is an asset reference for a TMMModel, and it references a file. Each of the types is defined in the following table. <pre> 1 <assetreference type="TMMModel"> 2 <file>greek\units\naval\juggernaut\juggernaut</file> 3 </assetreference> </pre>

The table below provides details about all the asset types that are recognized by the model system.

Asset Type	Description
TMAAnimation	- File (must point to a valid *.tma file). - Optional attribute "weight" (default 1.0f) to control blend weight. - Tag - Required Attributes: - type (required and must match a valid tag event type). - Value:

- Expected to be a float value to be used as a position.
- **Sound Attributes:**
 - checkVisible (optional - default is false) “1” to set true.
 - checkOwner (optional - default is false) “1” to set true.
 - looping (optional - default is false) “1” to set true.
 - set (optional - a sound set)
 - Example: `<tag type="SpecificSoundSet" checkvisible="1" set="TitanFootsteps">0.00</tag>`
- **Camera Shake Attributes**
 - force (optional - default is 0.0f)
 - duration (optional - default is 0.0f)
- **Destroy unit type data**
 - **unittype** (optional destroy unit type stuff ? unknown use).
 - **radius** (optional unknown)
- **Particles**
 - **popcornfxName** (optional - looks up the specific popcorn fx asset.
 - **bone** (optional for popcornfxName to specify a dummy bone index)
 - Use “bonethatdoesntexist” for anything that is deliberately not attached to a specific thing.
 - **movewithbone** (optional makes the particle move with the target bone when true) Defaults to true (“false” to set to false).
 - **stopwithanim** (optional makes the particle stop when the animation ends) Defaults to false (“true” to set to true).
 - **adjustduration** (optional changes the lifetime of the particle to the specified time).

- **popcornfxBloodFootprintName** (optional):

When the bloodied footprints setting is enabled and the unit is less than 80% health this effect name will be used instead of the popcornfx name.

- Example: `<tag type="particles" popcornfxname="titan\titan_foot_dust.pkfx" bone="vfx_foot_r" movewithbone="true" stopwithanim="false" adjustduration="false">0.52</tag>`

- **WindEffect**

- **effectName** (optional must match a valid effect name)
- **effectBone** (optional looks up dummy bones)

- **SpawnTempVisual** (spawns a visual with a short duration before despawn).

- **composite** - (required attribute that takes an engine path to a composite model to be spawned).
- **bone** - (optional - specifies a dummy bone to use for initial transform)
 - Use “bonethatdoesntexist” for anything that is deliberately not attached to a specific thing. This is the default behavior if attribute is not set.
- **duration** - (optional must be greater than 0 and less than 60, sets the lifetime for despawning the visual).

- Example: `<tag type="SpawnTempVisual" composite="vfx\weather\mist_vfx" duration="10.0">0.5</tag>`

- **component**

- Uses the text value between the component tags to determine the correct asset to use.

<component>COMPONENTNAMEHERE</component>

- weight
 - Weight for randomly choosing between anim assets (not a blend weight). Expects a long type value that is greater than 0.

TMMModel

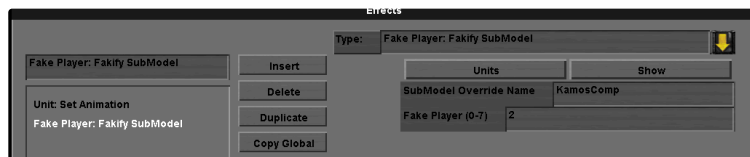
Expects a nested <File> tag referencing a *.tmm file.

```
1 <assetreference type="TMMModel">
2   <file>greek\units\naval\juggernaut\juggernaut</file>
3 </assetreference>
```

Warning! Do not reference a file that ends in "_damaged". You should always reference the version of a model that is not damaged. The destruction system will decide whether to show the damaged version automatically.

Other optional tags:

- **MaterialVariant** (sets the material variant index. Expects a whole number).
- **ModelVariant** (expects a valid sub model name)
- **PlayerColorOverrideName** (sets a name that can be used for overriding player color on the specific sub model by referencing a name set here).
 - "name" attribute is the name to use.



To use the player color override name just add in the tag as seen above where name is set to "KamosComp" for example. Then that can be referenced in triggers by referencing that specific name as part of the **Fake**

	Player: Fakify SubModel trigger. Warning this is recommended only for use during cinematics.
TMComposite	Expects a nested <File> tag with reference to a *.composite file in the same way it is done for a TMMModel. Other optional tags: • MaterialVariant (sets the material variant index. Expects a whole number). • PlayerColorOverrideName (sets a name that can be used for overriding player color on the specific sub model by referencing a name set here).
PopcornFX	Expects a nested <File> tag referencing a valid popcornfx file.

3.1.2 <logic>

Logic type tags require a type attribute. This is used to determine how the logic should be evaluated. Logic variations can lead to more logic variations or any other tag type expected to be within a component tag structure. E.g., an <assetreference>.

Logic Type	Description
Variation	Variation logic is selected using a random number. A collection of child nodes should be supplied, all defined as <variantX> where X is a number from 0 to N-1. A variant is expected to exist for every X sequentially from 0 to N-1 with no duplicates. The random number is, in the simplest case, used to select a variation by doing the following, where variationIndex is usually a random number. <pre>1 tempVariationIndex = variationIndex % weightSum;</pre> By default, the weight for each variation is 1. In this case, the variationIndex will directly map to the numbered variants. A custom weight can be specified

for each variation using a "weight" attribute with a number 1 or higher specified.

When weights are different for the variations, tempVariationIndex will be used to walk through all the variations linearly until the relative weighted position is found.

Simple Case Example:

```
1 <component>ModelComp
2   <logic type="Variation">
3     <variant0>
4       <submodelref ref="house_g_a"></submodelref>
5     </variant0>
6     <variant1>
7       <submodelref ref="house_g_b"></submodelref>
8     </variant1>
9     <variant2>
10      <submodelref ref="house_g_c"></submodelref>
11    </variant2>
12  </logic>
13 </component>
```

Civ	Civ refers to the specific god the player is playing. Evaluation of the Civ tag is dependent on detecting the associated player. When no player is found or there is no matching Civ for the player found, the default will always be the first child node. In most cases, Civ will only need to specify the gods for one Civ because Age of Mythology: Retold has the Culture types split into separate directories. The Civ names used for matching can be found in gameplay/major_gods.xml. Example: <pre> 1 <component>ModelComp 2 <logic type="Civ"> 3 <Zeus> 4 <submodelref ref="wonder_zeus"> 5 </submodelref> 6 </Zeus> 7 <Poseidon> 8 <submodelref ref="wonder_poseidon"> 9 </submodelref> 10 </Poseidon> 11 <Hades> 12 <submodelref ref="wonder_hades"> 13 </submodelref> 14 </Hades> 15 </logic> 16 </component> </pre>
Tech	Tech controls the output based on whether specific technologies have been learned. The result is the last defined tech that is currently active. Iteration goes from last to first and returns the first valid + active result. The names of tags should match tech names. Typically, you would find the first child will be called <none> to indicate the state that should be shown when no Tech is active. The default will always be the first child when no techs are active from the options. Example: <pre> 1 <component> 2 ModelComp 3 <logic type="Tech"> 4 <none> 5 <assetreference type="TMMModel"> 6 </assetreference> 7 </logic> 8 <file>greek\units\siege\helepolis\helepolis</file> 9 <materialvariant index="0"/> </pre>

```

8         </assetreference>
9     </none>
10    <Engineers>
11        <assetreference type="TMModel">
12
13    <file>greek\units\siege\helepolis\helepolis_upgrade
14    </file>
15        <materialvariant index="1"/>
16    </assetreference>
17    </Engineers>
18 </logic>
19 </component>

```

Age

The Age logic type chooses based on the player's current age. Logic values are expected to be listed in order of age. If a visual does not have an associated player or data necessary to determine an age, the default is the first child. Age elements are evaluated from last to first, and select the first child found to have an age greater than or equal to the current age.

Valid ages include (in the order):

- ArchaicAge
- ClassicalAge
- HeroicAge
- MythicAge

Example:

```

1 <component>
2     ModelComp
3     <logic type="Age">
4         <ArchaicAge>
5             <assetreference type="TMComposite">
6
7         <file>greek\buildings\town_center\town_center_g_age
8         1</file>
9             </assetreference>
10            </ArchaicAge>
11            <ClassicalAge>
12                <assetreference type="TMComposite">
13
14            <file>greek\buildings\town_center\town_center_g_age
15            2</file>
16                </assetreference>
17                </ClassicalAge>
18                <HeroicAge>
19                    <logic type="Tech">
20                        <none>
21                            <assetreference type="TMComposite">
22
23            <file>greek\buildings\town_center\town_center_g_age
24            3</file>
25                </assetreference>
26                </none>
27                <FortifiedTownCenter>
28                    <assetreference type="TMComposite">

```

```

23 <file>greek\buildings\town_center\town_center_g_for
    fortified_age3</file>
24     </assetreference>
25     </FortifiedTownCenter>
26 </logic>
27 </HeroicAge>
28 <MythicAge>
29     <logic type="Tech">
30         <none>
31         <assetreference type="TMComposite">
32
33 <file>greek\buildings\town_center\town_center_g_age
    4</file>
34     </assetreference>
35     </none>
36     <FortifiedTownCenter>
37         <assetreference type="TMComposite">
38
39 <file>greek\buildings\town_center\town_center_g_for
    fortified_age4</file>
40     </assetreference>
41     </FortifiedTownCenter>
42 </logic>
43 </MythicAge>
44 </logic>
45 </component>

```

BuildingCompletion

Building completion is primarily used for buildings that have not yet been fully built to show partial stages of construction. Tags are written as <pX> where the X is a whole number percent. <p0> would show from 0% building completion until the next interval. Most of the time intervals are defined as:

- <p0> (0% to < 33%)
- <p33> (33% to < 66%)
- <p66> (66% to < 100%)
- <p100> (100%)

If a valid lookup of the building percent can not be completed, the default will be the last child (should be 100%).

Example with both building completion AND age logic.

```

1 <component>
2     ModelComp
3     <logic type="BuildingCompletion">
4         <p0>
5             <assetreference type="TMComposite">
6
7 <file>shared\buildings\construction\construction_me
    dium_a</file>
8     </assetreference>
9     </p0>
10    <p33>

```

```

10         <assetreference type="TMComposite">
11         <file>shared\buildings\construction\construction_medium_b</file>
12         </assetreference>
13     </p33>
14     <p66>
15         <assetreference type="TMComposite">
16         <file>greek\buildings\stable\stable_g_const</file>
17         </assetreference>
18         </p66>
19         <p100>
20             <logic type="Age">
21                 <ArchaicAge>
22                     <assetreference
23 type="TMComposite">
24 <file>greek\buildings\stable\stable_g_age2</file>
25 </assetreference>
26 </ArchaicAge>
27 <ClassicalAge>
28 <assetreference
29 type="TMComposite">
30 <file>greek\buildings\stable\stable_g_age2</file>
31 </assetreference>
32 </ClassicalAge>
33 <HeroicAge>
34 <assetreference
35 type="TMComposite">
36 <file>greek\buildings\stable\stable_g_age3</file>
37 </assetreference>
38 </HeroicAge>
39 <MythicAge>
40 <assetreference
41 type="TMComposite">
42 <file>greek\buildings\stable\stable_g_age4</file>
43 </assetreference>
44 </MythicAge>
45 </logic>
46 </p100>
47 </logic>
48 </component>

```

Culture

Culture is different from Civ. It represents the broad "Greek" or "Norse" instead of the specific god the player has selected to play as. Each child tag must match a valid Culture name.

The first child is the default when a match can not be found for the player's current culture.

The Culture names used for matching can be found in **gameplay/cultures.xml**.

Example:

```

1 <component>

```

```

2      ModelComp
3      <logic type="Culture">
4          <Greek>
5              <assetreference type="TMComposite">
6
7          <file>spc\buildings\shrine\shrine_greek</file>
8              </assetreference>
9          </Greek>
10         <Egyptian>
11             <assetreference type="TMComposite">
12
13         <file>spc\buildings\shrine\shrine_egyptians</file>
14             </assetreference>
15         </Egyptian>
16         <Norse>
17             <assetreference type="TMComposite">
18
19         <file>spc\buildings\shrine\shrine_norse</file>
20             </assetreference>
21         </Norse>
22         <Atlantean>
23             <assetreference type="TMComposite">
24
25         <file>spc\buildings\shrine\shrine_atlantean</file>
26             </assetreference>
27         </Atlantean>
28     </logic>
29 </component>

```

Destruction

WARNING DEPRECATED!

Destruction logic is considered deprecated. If there is a particular need for this to be used, it may be safe to enable it. The problem with this logic type is that it results in the recomputation of visual assets in a destructive way that breaks the destruction systems. This logic is probably safe to use if there is a situation where the unit visual and ALL its children do not rely on using the destruction simulation.

Destruction logic works similarly to the building percent completion, except in reverse. Tags are defined as <pX> where normally at least a <p1> and <p99> are defined. The <p1> would be shown from 0% to less than 99%, and the <p99> would be shown at 99% or higher. Any number of additional intervals can be included. The evaluation is from the last child to the first child, so values should be sequential.

Example (from tamarisk_tree.xml):

```

1 <component>ModelComp
2   <logic type="Destruction">
3     <p0>

```

```

4         <submodelref ref="stage_complete"/>
5     </p0>
6     <p1>
7         <submodelref ref="stage3"/>
8     </p1>
9     <p25>
10        <submodelref ref="stage3"/>
11    </p25>
12    <p50>
13        <submodelref ref="stage2"/>
14    </p50>
15    <p75>
16        <submodelref ref="stage1"/>
17    </p75>
18 </logic>
19 </component>

```

Inventory

The inventory logic type can show a visual relative to the quantity of resources remaining in a unit's inventory. It will only work correctly for units that have a carry capacity.

Example (from berry_bush.xml):

```

1 <logic type="Variation">
2   <variant0>
3     <logic type="Inventory">
4       <p0>
5         <assetreference type="TMComposite">
6
7         <file>nature\bushes\berry_bush\berry_bush_a\berry_bush_a</file>
8         <materialvariant index="2"/>
9         </assetreference>
10      </p0>
11      <p33>
12        <assetreference type="TMComposite">
13
14        <file>nature\bushes\berry_bush\berry_bush_a\berry_bush_a</file>
15        <materialvariant index="1"/>
16        </assetreference>
17      </p33>
18      <p66>
19        <assetreference type="TMComposite">
20
21        <file>nature\bushes\berry_bush\berry_bush_a\berry_bush_a</file>
22        <materialvariant index="0"/>
23        </assetreference>
24      </p66>
25    </logic>
26  </variant0>
27  <variant1>
28    <logic type="Inventory">
29      <p0>
30        <assetreference type="TMComposite">
31
32        <file>nature\bushes\berry_bush\berry_bush_b\berry_bush_b</file>
33        <materialvariant index="2"/>
34        </assetreference>
35      </p0>
36      <p33>

```

```

33         <assetreference type="TMComposite">
34         <file>nature\bushes\berry_bush\berry_bush_b\berry_b
ush_b</file>
35             <materialvariant index="1"/>
36             </assetreference>
37         </p33>
38         <p66>
39             <assetreference type="TMComposite">
40             <file>nature\bushes\berry_bush\berry_bush_b\berry_b
ush_b</file>
41                 <materialvariant index="0"/>
42                 </assetreference>
43             </p66>
44         </logic>
45     </variant1>
46     <variant2>
47         <logic type="Inventory">
48             <p0>
49                 <assetreference type="TMComposite">
50                 <file>nature\bushes\berry_bush\berry_bush_c\berry_b
ush_c</file>
51                     <materialvariant index="2"/>
52                     </assetreference>
53                 </p0>
54                 <p33>
55                     <assetreference type="TMComposite">
56                     <file>nature\bushes\berry_bush\berry_bush_c\berry_b
ush_c</file>
57                         <materialvariant index="1"/>
58                         </assetreference>
59                     </p33>
60                     <p66>
61                         <assetreference type="TMComposite">
62                         <file>nature\bushes\berry_bush\berry_bush_c\berry_b
ush_c</file>
63                             <materialvariant index="0"/>
64                             </assetreference>
65                         </p66>
66                     </logic>
67                 </variant2>
68             </logic>

```

UnitType

Changes the appearance depending on the type of unit. This logic can be used to create generic effect files that modify how the effect is displayed, depending on the type of unit.

If no type can be matched, the default is to return the first child (typically <none>). This will not work for any UIVisual. Any visual with a valid unit will evaluate from the first index and return either the first valid child matching the unit's type, the index of the last invalid unit type (expected to be <none>), or index 0.

Example (from shrine_active.xml):

```

1 <component>glow
2   <logic type="UnitType">
3     <LogicalTypeAnimalSmall>
4       <assetreference type="TMComposite">
5
6     <file>vfx\glow\shrine_active_small</file>
7       </assetreference>
8     </LogicalTypeAnimalSmall>
9     <LogicalTypeAnimalLarge>
10       <assetreference type="TMComposite">
11
12     <file>vfx\glow\shrine_active_large</file>
13       </assetreference>
14     </LogicalTypeAnimalLarge>
15     <GoldResource>
16       <assetreference type="TMComposite">
17
18     <file>vfx\glow\shrine_active_large</file>
19       </assetreference>
20     </GoldResource>
21     <BerryBush>
22       <assetreference type="TMComposite">
23
24     <file>vfx\glow\shrine_active_large</file>
25       </assetreference>
26     </BerryBush>
27     <Tree>
28       <nullvisual/>
29     </Tree>
30     <Resource>
31       <assetreference type="TMComposite">
32
33     <file>vfx\glow\shrine_active_small</file>
34       </assetreference>
35     </Resource>
36   </logic>
37 </component>

```

Skin

Populates the model by looking up skin data and attaching all required components based on the skinId. The default case matching no specific skin should be the first child node.

Currently unused in Age of Mythology: Retold.

CarryType

Evaluates based on the type of resource held. The default case can be named <none> and will be used if the unit's inventory does not have any defined resources with a non-zero quantity.

Example:

```

1 <component>
2   ModelComp
3     <Logic type="CarryType">
4       <none>
5         <assetreference type="TMModel">
6
7     <file>egyptian\units\economic\caravan_egyptian\caravan_egyptian</file>

```

	<pre> 7 </assetreference> 8 </none> 9 <gold> 10 <assetreference type="TMModel"> 11 <file>egyptian\units\economic\caravan_egyptian\cara van_egyptian_loaded</file> 12 </assetreference> 13 </gold> 14 </Logic> 15 </component> </pre>
Contain	Contain logic is binary, dependent on whether the unit contains other units. The only valid tags are <true> and <false>.
Cinematic	Cinematic logic only takes two possible tags. <cinematic> and <normal>. The <normal> option is used except when the cinematic mode is enabled. Example: <pre> 1 <logic type="Cinematic"> 2 <normal> 3 <logic type="Tech"> 4 <none> 5 <assetreference type="TMModel"> 6 <file>greek\units\infantry\hoplite\hoplite_iron</fi le> 7 </assetreference> 8 </none> 9 <MediumInfantry> 10 <assetreference type="TMModel"> 11 <file>greek\units\infantry\hoplite\hoplite_bronze</ file> 12 </assetreference> 13 </MediumInfantry> 14 <HeavyInfantry> 15 <assetreference type="TMModel"> 16 <file>greek\units\infantry\hoplite\hoplite_silver</ file> 17 </assetreference> 18 </HeavyInfantry> 19 <ChampionInfantry> 20 <assetreference type="TMModel"> 21 <file>greek\units\infantry\hoplite\hoplite_gold</fi le> 22 </assetreference> 23 </ChampionInfantry> 24 </logic> 25 </normal> 26 <Cinematic> 27 <assetreference type="TMModel"> 28 <file>greek\units\infantry\hoplite\hoplite_silver</ file> 29 </assetreference> 30 </Cinematic> </pre>

Veterancy

Veterancy is used for specific units like the Hydra that change depending on combat as they gain combat experience. Units with support for gaining experience have a veterancy rank that is used to determine the version that is supposed to be shown.

Each rank is represented using <rX>, where X represents the rank from 0 to N-1. The exact rank is looked up. The visual will default to the first index if a valid rank is not found. The default case when no veterancy is gained is recognized by a <none> tag.

Example:

```
1 <component>
2   ModelComp
3   <logic type="Veterancy">
4     <none>
5       <assetreference type="TMModel">
6
7     <file>greek\units\myth\hydra\hydra_1head</file>
8     </assetreference>
9     </none>
10    <r0>
11      <assetreference type="TMModel">
12
13    <file>greek\units\myth\hydra\hydra_2head</file>
14    </assetreference>
15    </r0>
16    <r1>
17      <assetreference type="TMModel">
18
19    <file>greek\units\myth\hydra\hydra_3head</file>
20    </assetreference>
21    </r1>
22    <r2>
23      <assetreference type="TMModel">
24
25    <file>greek\units\myth\hydra\hydra_4head</file>
26    </assetreference>
27    </r2>
28    <r3>
29      <assetreference type="TMModel">
30
31    <file>greek\units\myth\hydra\hydra_5head</file>
32    </assetreference>
33    </r3>
34  </logic>
35 </component>
```

Tier

Represents a tier of unit and behaves in a similar way to the above. The difference is that the “Modify Tier” property is used instead of the “Veterancy Rank”.

	Currently unused in Age of Mythology: Retold.
Editor	Accepts <normal> and <editor> as valid tags. <editor> content shows while in the editor, and <normal> shows in all other situations. Example: <pre> 1 <component>ModelComp 2 <logic type="Editor"> 3 <normal> 4 <nullvisual /> 5 </normal> 6 <editor> 7 <assetreference type="TModel"> 8 <file>misc\editor\3d_icons\light_icon\light_icon</file> 9 </assetreference> 10 </editor> 11 </logic> 12 </component> </pre>
MaxGatherers	Accepts <true> and <false> as valid tags. Will use true when the max gatherers are assigned to a unit and false in all other situations. Example: <pre> 1 <component>ModelComp 2 <logic type="MaxGatherers"> 3 <false> 4 <nullvisual/> 5 </false> 6 <true> 7 <assetreference type="TModel"> 8 <file>shared\ui\fishing_buoy\fishing_buoy</file> 9 </assetreference> 10 </true> 11 </logic> 12 </component> </pre>
AutoCommand	Accepts <true> and <false> as valid tags. Will use True when the unit has an auto command. Example: <pre> 1 <component>VFXAutoComp 2 <logic type="AutoCommand"> 3 <false> 4 <nullvisual/> 5 </false> 6 <true> 7 <assetreference type="PopcornFx"> 8 <file>glow/skypassage/skypassage_symbol_glow.pkfx</file> </pre>

	<pre> 9 file> 10 </assetreference> 11 </true> 12 </logic> 13 </component> </pre>
Theme	Used for special events to swap visuals thematically. This can be overridden by players using a setting to force visuals back to their defaults. Currently, the only event using this is Thanksgiving. Accepts <none> as the default case, and other cases like <thanksgiving> will only work if the event is currently active at the time of playing a match. Example: <pre> 1 <component>ModelComp 2 <logic type="Theme"> 3 <none> 4 <submodelref ref="normal_chicken"/> 5 </none> 6 <thanksgiving> 7 <submodelref ref="event_turkey"/> 8 </thanksgiving> 9 </logic> 10 </component> </pre>
Training	Changes visuals based on whether the unit is currently training or researching something. Accepts <true> and <false> as valid tags. Example: <pre> 1 <logic type="Training"> 2 <false> 3 <assetreference type="TMComposite"> 4 <file>vfx\\glow\\shinboku_idle</file> 5 </assetreference> 6 </false> 7 <true> 8 <assetreference type="TMComposite"> 9 <file>vfx\\glow\\shinboku_activate</file> 10 </assetreference> 11 </true> 12 </logic> </pre>
SizeClass	Uses the SizeClass defined in ProtoUnits to determine which visual to use. Accepts <sN> where N is the size class number. The logic will check for the head unit if there is one and use its size class if available. The sizes should be ordered from largest to smallest for any

values that you need. The first value where the units SizeClass is >= the logic value will be selected.

Example:

```
1 <logic type="SizeClass">
2   <s8>
3     <assetreference type="PopcornFx">
4       <file>godpowers\\divine_shield\\divine_shield_large
        .pkfx</file>
5     </assetreference>
6   </s8>
7   <s7>
8     <assetreference type="PopcornFx">
9       <file>godpowers\\divine_shield\\divine_shield.pkfx<
        /file>
10    </assetreference>
11  </s7>
12  <s0>
13    <assetreference type="PopcornFx">
14      <file>godpowers\\divine_shield\\divine_shield_small
        .pkfx</file>
15    </assetreference>
16  </s0>
17 </logic>
```

AssetPack

The AssetPack logic type allows tags to match <None> as the default, and otherwise valid asset pack names. This is used to display the golden statues for the supporter pack.

DLCTheme

The DLCTheme logic type allows tags to match <None> as the default, and otherwise valid DLC names. This is used for the BloodPack DLC to show unique visuals.

3.1.3 <attach>

Attach Attribute	Description
a OR name	You can use either "a" or "name" to reference the named attachment. "a" takes precedence and will cause "name" to be ignored. They have the same meaning.
fromBone	If no fromBone is specified, it is assumed that the model's root is used (0, 0, 0).

	You can optionally also write “ bonethatdoesntexist ” to get the same behavior.
toBone OR attachpoint	toBone and attachpoint are interchangeable. toBone takes precedence and will cause attachpoint to be ignored. They have the same meaning. This position tracks where the attachment is moving to while animations play. If you are deliberately naming an invalid bone use “bonethatdoesntexist” as the name.
syncAnims	A boolean is specified by using the values "0" or "1". The default is false, so the only time to specify this attribute is if you want to syncAnims by specifying "1" as the attribute value.
allowRotation	A boolean is specified by using the values "0" or "1". Default is false, so the only time to specify this attribute is if you want to allowRotation by specifying "1" as the attribute value.
keepDuringDestruction	A boolean is specified by using the values "0" or "1". Default is false, so the only time to specify this attribute is if you want to keepDuringDestruction by specifying "1" as the attribute value. When set to 1 it will prevent the default behavior of hiding visuals not related directly to destruction for the specific attachment. A use case for this is when something spawns in as part of the destruction, such as Kraken tentacles.

3.1.4 <submodelref>

Submodelref tags only have one attribute, "ref", specifying a named submodel. The below example demonstrates how this can be used to separate the definition of models from the variation logic.

Example:

```
1 <submodel>house_g_a
```

```

2   <include>greek\buildings\house\house_g_a.xml</include>
3 </submodel>
4 <submodel>house_g_b
5   <include>greek\buildings\house\house_g_b.xml</include>
6 </submodel>
7 <submodel>house_g_c
8   <include>greek\buildings\house\house_g_c.xml</include>
9 </submodel>
10 <component>ModelComp
11   <logic type="Variation">
12     <variant0>
13       <submodelref ref="house_g_a"></submodelref>
14     </variant0>
15     <variant1>
16       <submodelref ref="house_g_b"></submodelref>
17     </variant1>
18     <variant2>
19       <submodelref ref="house_g_c"></submodelref>
20     </variant2>
21   </logic>
22 </component>

```

3.2 Anim Tag

The name of the anim must be a valid anim type to be loaded.

Example Anim:

```

1 <anim>Idle
2   <assetreference type="TMAAnimation">
3     <file>greek\units\naval\juggernaut\anim\juggernaut_idle_a</file>
4   </assetreference>
5   <component>ModelComp</component>
6 </anim>

```

The above example has an Idle-type animation and combines an animation file with a component already defined elsewhere in the file. The following table defines the attributes/tags you can use inside an Anim tag.

Tag	Description
Attribute: AnimLengthOverride	This overrides the length of the animation to be the specified value. (expects a float).
Attribute: includeAttachmentsInBounds	This is used during Sim Database generation to force the inclusion of additional sub-components in the units selection bounds. Accepts "true" or "1" to enable this feature.
Attribute: padVisualBound	Pads the Min bounds side of the bounding box used for visual culling. This can be useful if animations travel far

sMinXYZ	from the unit's origin.
Attribute: padVisualBound sMaxXYZ	Pads the Max bounds side of the bounding box used for visual culling. This can be useful if animations travel far from the unit's origin.
<assetreference >	Can contain anything that the <assetreference> tag can contain; as defined earlier in this document. See section 3.1.1 <assetreference> for more details.
<component>	Searches for an already existing <component> tag by the name specified between the tags.
<attach>	Supports all the same data as the attach tag that can be used from within a component tag. See section 3.1.3 <attach> for more details.

3.3 Attachment and Submodel Tags

Both the attachment and submodel tags work in very similar ways. They go through the same code for parsing, with the only difference being which data set they reference for storing the linking data.

Tag Options	Description
<include>	A file reference to the included XML file that will be used to generate a model. Example: <pre> 1 <submodel>house_g_a 2 <include>greek\buildings\house\house_g_a.xml</include> 3 </submodel></pre>
Model Data	An entire structure of nested model data. Example: <pre> 1 <attachment>KrakenTentacle1 2 <component> <!-- Be mindful of this name we can get namespace conflicts if it shares the same name with other components --> 3 KrakenTentacle1Comp 4 <assetreference type="TMModel"> 5 <file>norse\units\myth\kraken\kraken_tentacle_lrg</fi le> 6 </assetreference></pre>

```

7     </component>
8
9     <anim>KrakenDeath
10         <assetreference type="TMAAnimation">
11
12             <file>norse\units\myth\kraken\anim\Kraken_tentacle_1r
13             g_wrapTest</file>
14         </assetreference>
15         <component>KrakenTentacle1Comp</component>
16     </anim>
17 </attachment>

```

4 Animation Types

The following list defines all Animation Types that Age of Mythology: Retold currently accepts. Some of these are deprecated or unused.

"None"
"Attack"
"Idle"
"Bored"
"Build"
"Carry"
"Death"
"Decay"
"GatherChop"
"GatherFarm"
"GatherHunt"
"GatherMine"
"RangedAttack"
"Walk"
"PivotRight"
"PivotLeft"
"Birth"

"Flai"
"Cinematic"
"Pickup"
"Empower"
"Convert"
"Opened"
"Closed"
"Opening"
"Closing"
"GatherBush"
"TeleportIn"
"TeleportOut"
"GatherFish"
"Heal"
"Recover"
"Burning"
"BurnDeath"
"Destruct"
"Recoil"
"Jog"
"Run"
"death_by_havoc"
"death_havoc_flail"
"death_havoc_spin"
"Gore"

"Pray"
"Capsize"
"Eat"
"JumpFly"
"Charging"
"MaxLOS"
"CarryIdle"
"CarryBored"
"KrakenDeath"
"BurstHeal"
"HeroizeUp"
"HeroizeFly"
"HeroizeDown"
"FlyDown"
"FlyUp"
"AoESpinning"
"BirthWater"
"BirthLand"